Sơ đồ tuần tự: Tổng thể và từng lớp

VIB Internal Knowledge Assistant (bot RAG trên Slack)

COS40005 Computing Technology Project A · Swinburne University of Technology
Bản để nhóm xem xét · 31/10/2026

Trạng thái: bản nháp để nhóm thảo luận. Mục ghi "Giả định thiết kế" hoặc "Đề xuất" chưa được thống nhất với khách hàng.

Mục lục

1. Luồng hỏi–đáp tổng thể (từ đầu đến cuối) [Tổng thể]
2. Luồng nạp dữ liệu tổng thể (Confluence vào kho tri thức) [Lớp 4]
3. Lớp 1: Presentation (giao diện Slack) [Lớp 1]
4. Lớp 2: Security và xác thực [Lớp 2]
5. Lớp 3: RAG core và logic nghiệp vụ [Lớp 3]
6. Lớp 4: Nạp dữ liệu chi tiết (offline) [Lớp 4]
7. Lớp 4: Tìm kiếm và cache lúc có câu hỏi [Lớp 4]
8. Các giả định thiết kế cần nhóm xác nhận

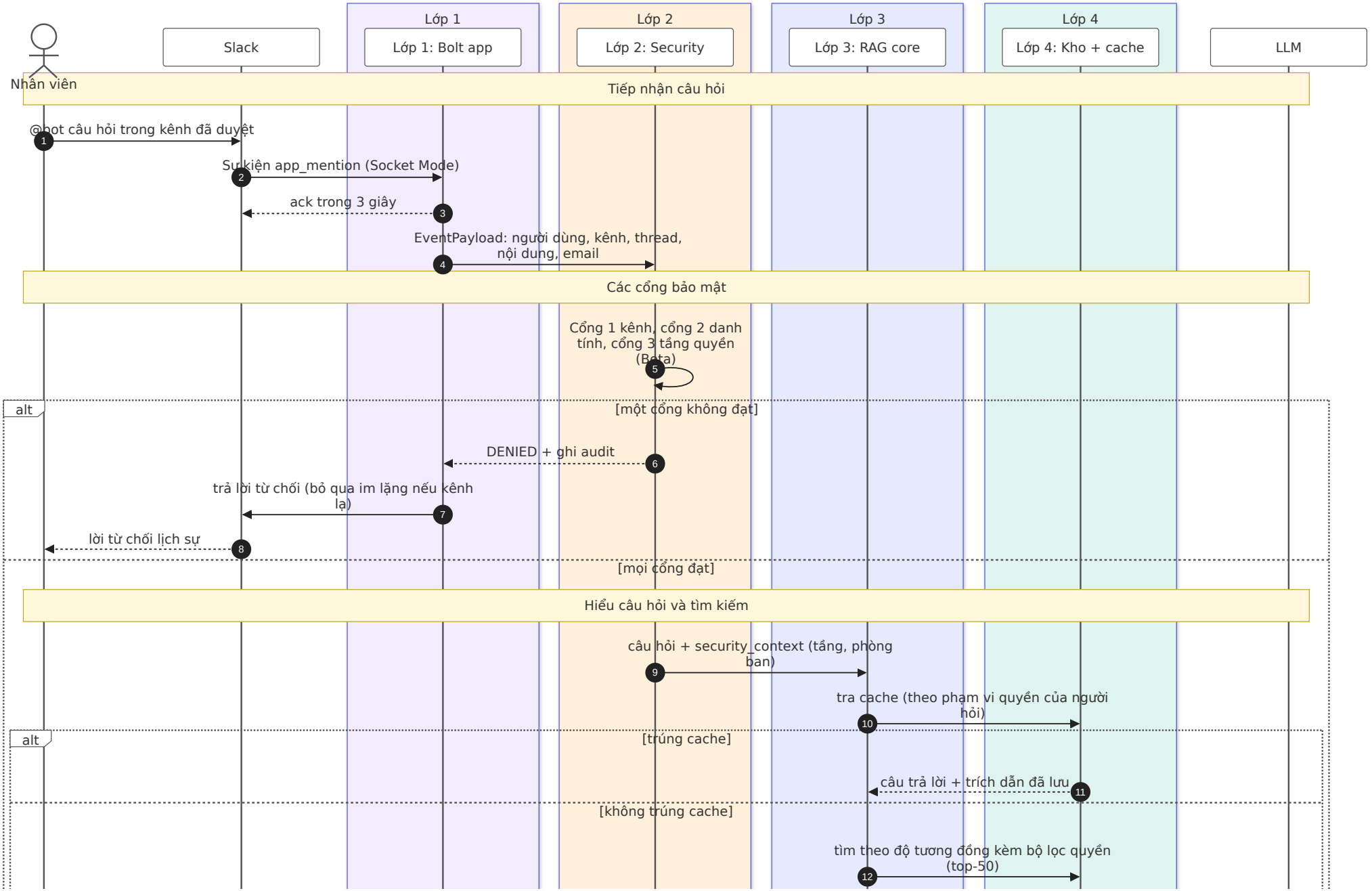
Cách đọc sơ đồ

- Mỗi đường dọc là một đối tượng tham gia. Thời gian chạy từ trên xuống dưới.
- Mũi tên liền = yêu cầu hoặc lời gọi. Mũi tên đứt = phản hồi. Mũi tên vòng lại chính nó = việc xử lý bên trong.
- Số trong vòng tròn là số bước. Các bảng sau mỗi sơ đồ dùng cùng số này.
- Khung: alt = một trong hai nhánh xảy ra (nếu / nếu không), opt = tùy chọn, chỉ khi điều kiện đúng, loop = lặp lại cho từng mục.
- Thanh vàng = các giai đoạn của luồng. Nền màu = bốn lớp (L1 tím, L2 cam, L3 xanh dương, L4 xanh lục).
- (Beta) = dự kiến cho bản Beta, không bắt buộc ở Alpha. “Giả định thiết kế” = do người vẽ sơ đồ thêm vào, cần nhóm xác nhận.

Lưu ý đánh số: số bước bắt đầu lại ở mỗi sơ đồ. Sơ đồ hỏi–đáp 30 bước trước đó được thay bằng sơ đồ 1 cùng các sơ đồ lớp từ 3 đến 7.

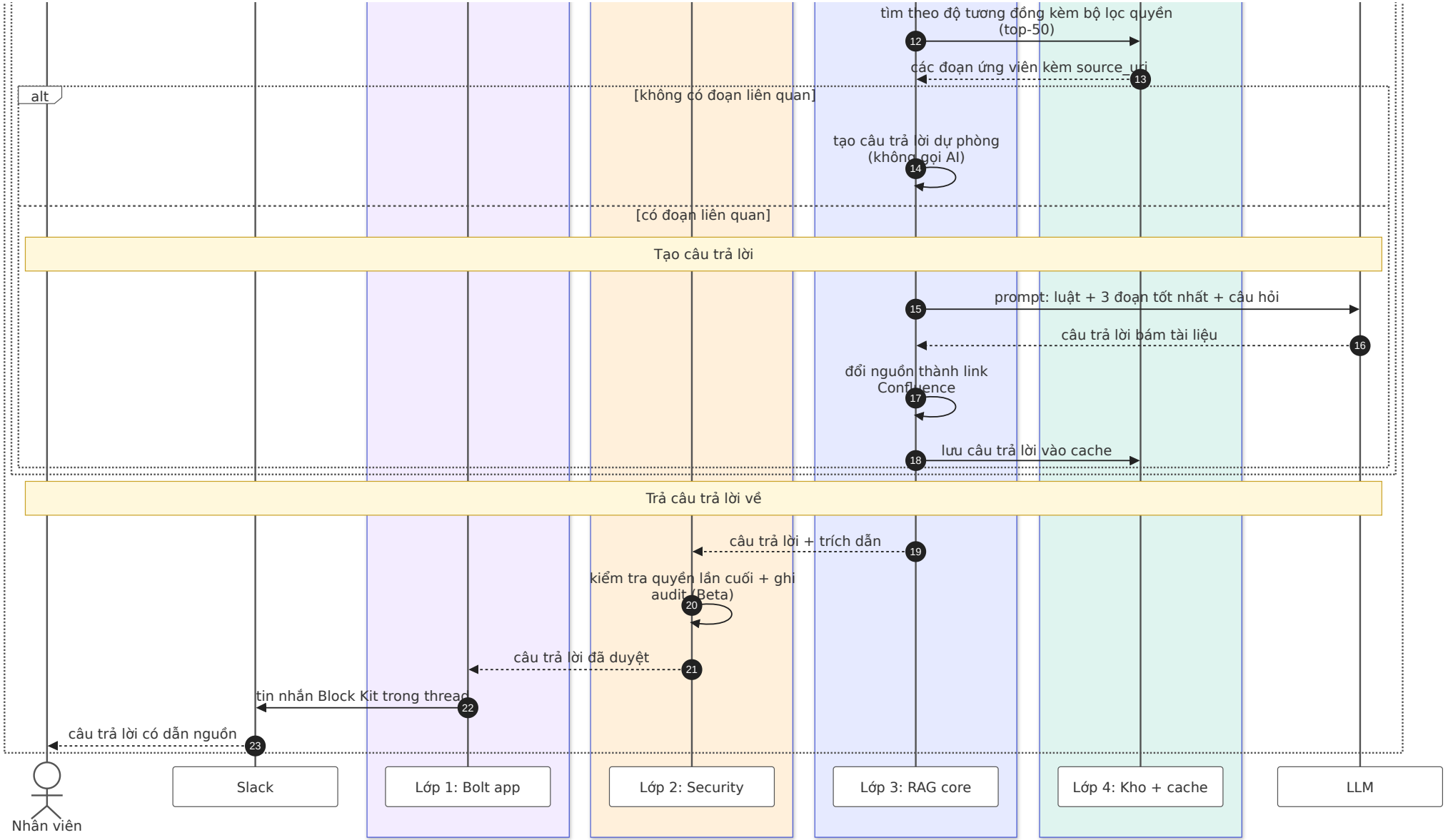
Tổng thể

Sơ đồ 1: Luồng hỏi–đáp tổng thể (từ đầu đến cuối) (1/2)



Tổng thể

Sơ đồ 1: Luồng hỏi-đáp tổng thể (từ đầu đến cuối) (2/2)



Sơ đồ 1: Luồng hỏi–đáp tổng thể (từ đầu đến cuối)

Tổng thể

Mục đích

Một câu hỏi đi qua cả bốn lớp rồi quay về thành câu trả lời có dẫn nguồn. Mỗi lớp được vẽ thành một đối tượng, để bạn thấy toàn bộ hành trình trong một trang. Các sơ đồ 3 đến 7 mở từng lớp ra để xem bên trong làm gì.

Các đối tượng tham gia

Đối tượng	Vai trò
Nhân viên	Đặt câu hỏi trong Slack
Slack	Nền tảng chat (bên ngoài)
Lớp 1: Bolt app	Nhận sự kiện, hiển thị câu trả lời
Lớp 2: Security	Các cổng, ngữ cảnh quyền, audit
Lớp 3: RAG core	Hiểu, tìm, tạo câu trả lời
Lớp 4: Kho + cache	Kho tri thức tìm kiếm được và cache
LLM	Mô hình AI (Ollama hoặc Bedrock)

Từng bước

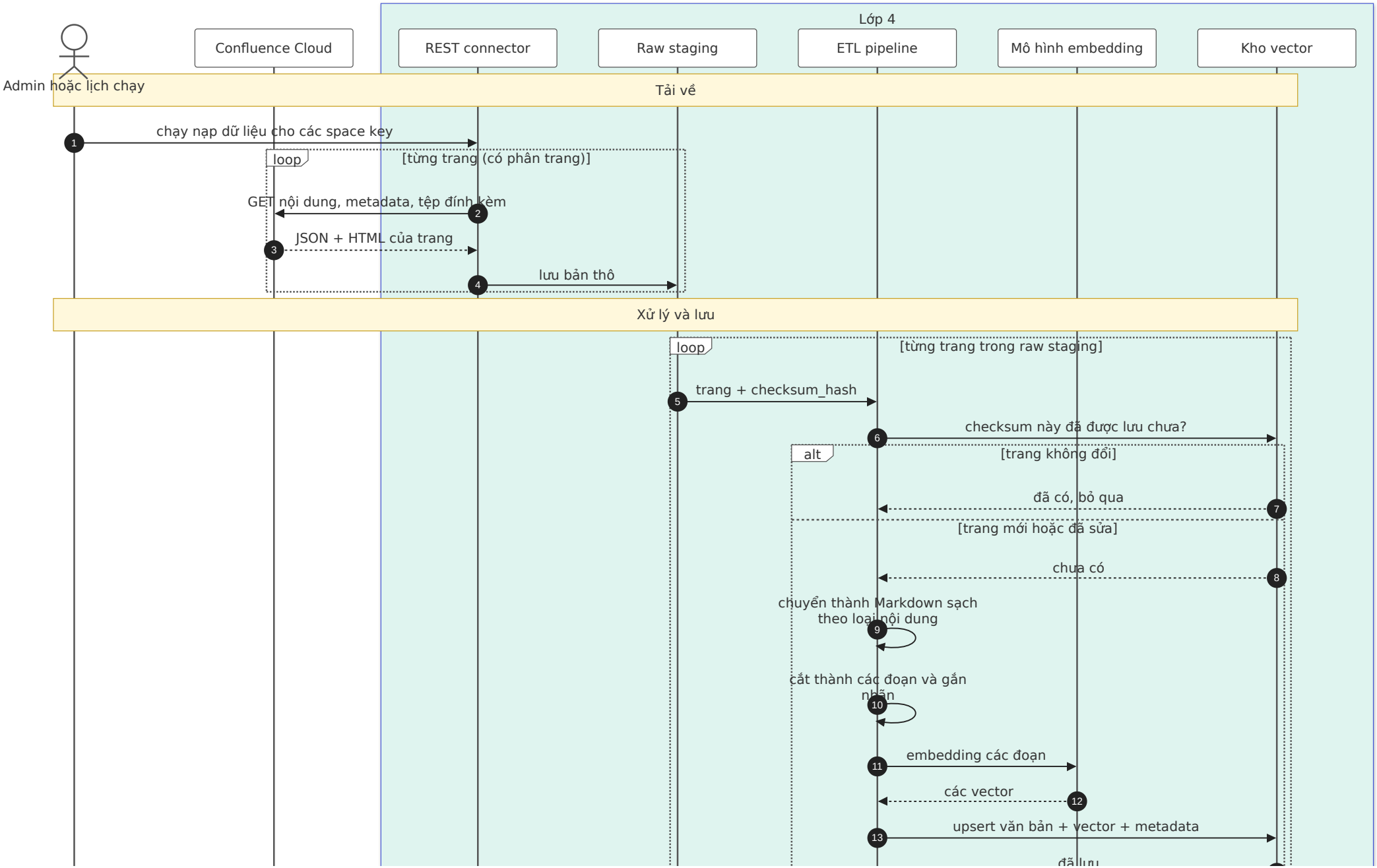
#	Từ → Đến	Điều gì xảy ra	Vì sao / ghi chú
Tiếp nhận câu hỏi			
1	Nhân viên → Slack	@bot câu hỏi trong kênh đã duyệt	Bot chỉ phản ứng khi được nhắc tên, không đọc các tin nhắn khác.
2	Slack → Lớp 1: Bolt app	Sự kiện app_mention (Socket Mode)	Slack đẩy sự kiện qua WebSocket do bot mở sẵn, nên không cần địa chỉ công khai.
3	Lớp 1: Bolt app → Slack	↩ ack trong 3 giây	Phản hồi chậm hơn 3 giây thì Slack coi là lỗi và gửi lại sự kiện.
4	Lớp 1: Bolt app → Lớp 2: Security	EventPayload: người dùng, kênh, thread, nội dung, email	Lớp 1 chỉ thu thập dữ kiện. Mọi quyết định về quyền do Lớp 2 đưa ra.
Các cổng bảo mật			
5	Lớp 2: Security ⟲	Cổng 1 kênh, cổng 2 danh tính, cổng 3 tầng quyền (Beta)	Các kiểm tra chạy theo thứ tự cố định, kiểm tra rẻ nhất chạy trước.
alt: một cổng không đạt			
6	Lớp 2: Security → Lớp 1: Bolt app	↩ DENIED + ghi audit	Không tìm kiếm gì và không gửi gì cho AI.
7	Lớp 1: Bolt app → Slack	trả lời từ chối (bỏ qua im lặng nếu kênh lạ)	Kênh lạ bị bỏ qua, không trả lời, để bot không lộ sự tồn tại. Người bị từ chối nhận một thông báo ngắn, lịch sự. (Giả định thiết kế.)
8	Slack → Nhân viên	↩ lời từ chối lịch sự	
else: mọi cổng đạt			
Hiểu câu hỏi và tìm kiếm			
9	Lớp 2: Security → Lớp 3: RAG core	câu hỏi + security_context (tầng, phòng ban)	security_context đi kèm câu hỏi và quyết định bộ lọc tìm kiếm.

#	Từ → Đến	Điều gì xảy ra	Vì sao / ghi chú
10	Lớp 3: RAG core → Lớp 4: Kho + cache	tra cache (theo phạm vi quyền của người hỏi)	Câu trả lời trong cache chỉ dùng lại cho người có cùng phạm vi quyền. Nếu không, câu trả lời của quản lý có thể lộ cho nhân viên. (Giả định thiết kế.)
alt: trúng cache			
11	Lớp 4: Kho + cache → Lớp 3: RAG core	↩ câu trả lời + trích dẫn đã lưu	Bỏ qua tìm kiếm, xếp hạng lại và lệnh gọi AI.
else: không trúng cache			
12	Lớp 3: RAG core → Lớp 4: Kho + cache	tìm theo độ tương đồng kèm bộ lọc quyền (top-50)	Bộ lọc nằm trong truy vấn, nên văn bản không được phép sẽ không bao giờ được trả về.
13	Lớp 4: Kho + cache → Lớp 3: RAG core	↩ các đoạn ứng viên kèm source_uri	
alt: không có đoạn liên quan			
14	Lớp 3: RAG core ⟲	tạo câu trả lời dự phòng (không gọi AI)	Nói “không tìm thấy” an toàn hơn là đoán.
else: có đoạn liên quan			
Tạo câu trả lời			
15	Lớp 3: RAG core → LLM	prompt: luật + 3 đoạn tốt nhất + câu hỏi	Luật: chỉ trả lời từ ngữ cảnh, trích dẫn mọi nguồn, trả lời theo ngôn ngữ của câu hỏi.
16	LLM → Lớp 3: RAG core	↩ câu trả lời bám tài liệu	
17	Lớp 3: RAG core ⟲	đổi nguồn thành link Confluence	
18	Lớp 3: RAG core → Lớp 4: Kho + cache	lưu câu trả lời vào cache	Chỉ câu trả lời có nguồn mới được cache, câu trả lời dự phòng thì không.
Trả câu trả lời về			
19	Lớp 3: RAG core → Lớp 2: Security	↩ câu trả lời + trích dẫn	Cũng dùng cho câu trả lời lấy từ cache và câu trả lời dự phòng (không có trích dẫn).
20	Lớp 2: Security ⟲	kiểm tra quyền lần cuối + ghi audit (Beta)	Mọi tài liệu được trích dẫn phải nằm trong quyền của người hỏi. Mỗi yêu cầu ghi một dòng log JSON.
21	Lớp 2: Security → Lớp 1: Bolt app	↩ câu trả lời đã duyệt	
22	Lớp 1: Bolt app → Slack	tin nhắn Block Kit trong thread	Thẻ trả lời có một nút cho mỗi nguồn.
23	Slack → Nhân viên	↩ câu trả lời có dẫn nguồn	

Điểm chính

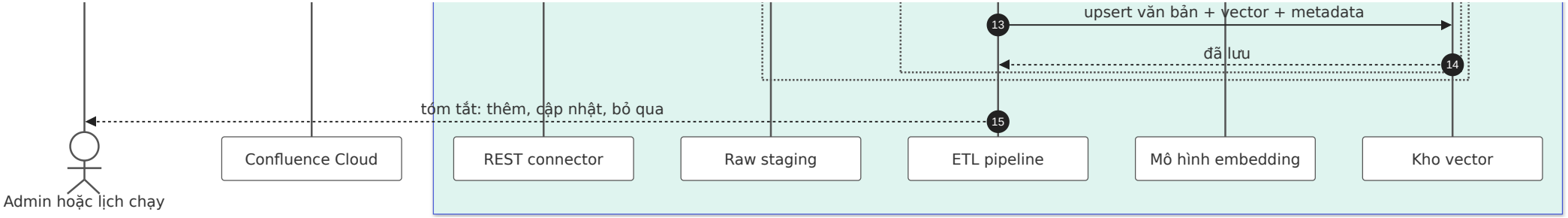
- Mọi câu hỏi đều qua ba cổng bảo mật trước khi tìm kiếm. Không có đường tắt từ Slack đến AI.
- Bộ lọc quyền nằm ngay trong câu truy vấn cơ sở dữ liệu, nên văn bản người hỏi không được xem sẽ không bao giờ được trả về hay đến tay AI.
- Có ba đường thoát sớm: bị từ chối (bước 6 đến 8), trúng cache (bước 11) và không tìm thấy gì (bước 14). Chỉ nhánh cuối của phần tìm kiếm mới gọi mô hình AI.
- Mục ghi Beta (kiểm tra tầng quyền, audit log) có thể tắt mà luồng Alpha vẫn chạy.

Sơ đồ 2: Luồng nạp dữ liệu tổng thể (Confluence vào kho tri thức) (1/2)



Lớp 4

Sơ đồ 2: Luồng nạp dữ liệu tổng thể (Confluence vào kho tri thức) (2/2)



Sơ đồ 2: Luồng nạp dữ liệu tổng thể (Confluence vào kho tri thức)

Lớp 4

Mục đích

Sơ đồ này cho thấy điều xảy ra trước khi có ai đặt câu hỏi: các trang Confluence được tải về, làm sạch, cắt nhỏ, gắn nhãn ai được xem, đổi thành vector và lưu lại. Quá trình chạy offline, do admin hoặc lịch chạy khởi động. Sơ đồ 6 vẽ cùng luồng này chi tiết hơn.

Các đối tượng tham gia

Đối tượng	Vai trò
Admin hoặc lịch chạy	Khởi động một lần nạp
Confluence Cloud	Nguồn trang tài liệu (bên ngoài)
REST connector	Tải trang qua API
Raw staging	Giữ bản gốc đã tải
ETL pipeline	Phân tích, cắt đoạn, gắn nhãn
Mô hình embedding	Đổi văn bản thành vector
Kho vector	PostgreSQL với pgvector

Từng bước

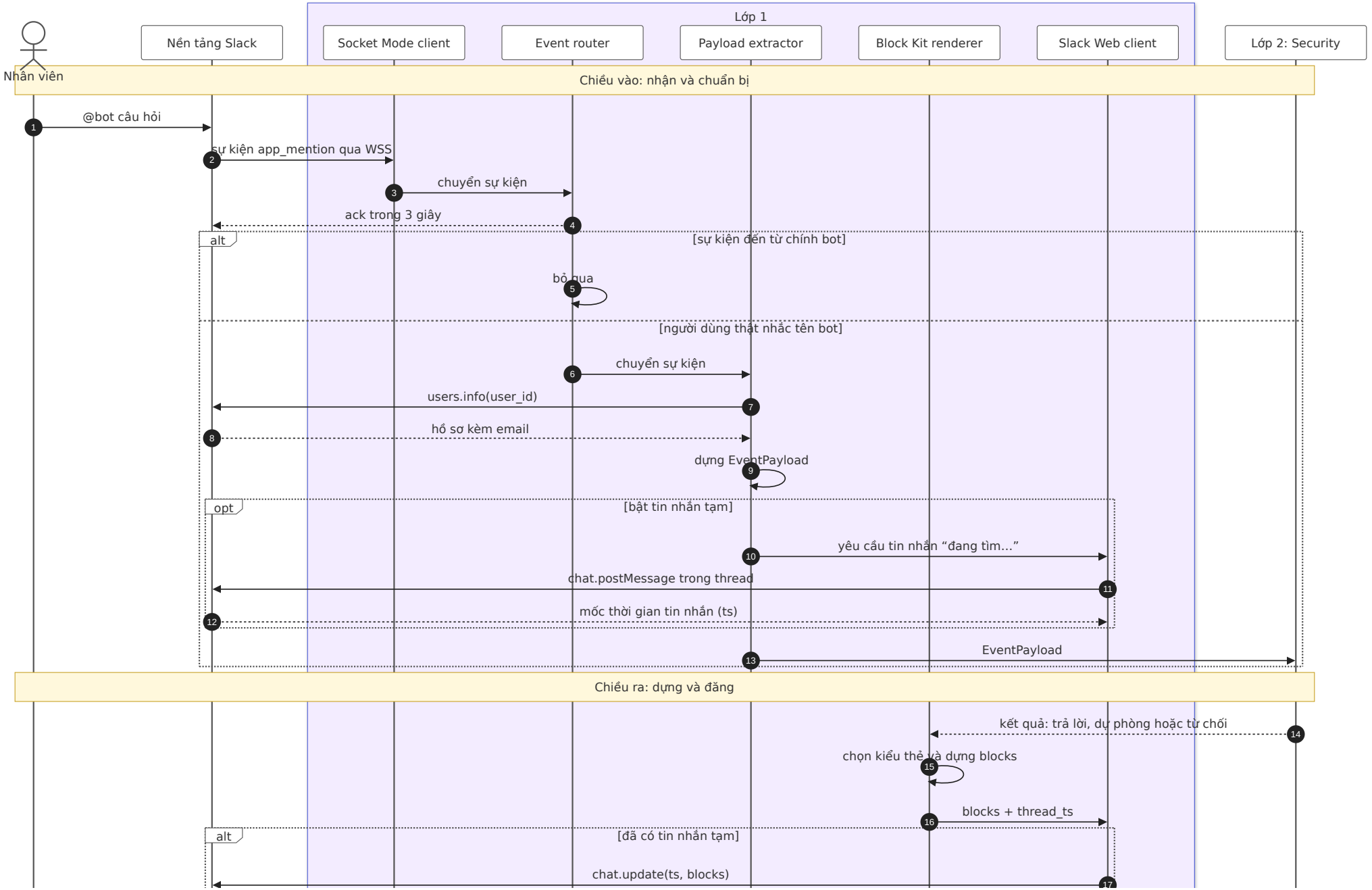
#	Từ → Đến	Điều gì xảy ra	Vì sao / ghi chú
Tải về			
1	Admin hoặc lịch chạy → REST connector	chạy nạp dữ liệu cho các space key	Alpha dùng một space mẫu VIB-Mock-Knowledge. Có thể chạy tay hoặc theo lịch.
loop: từng trang (có phân trang)			
2	REST connector → Confluence Cloud	GET nội dung, metadata, tệp đính kèm	
3	Confluence Cloud → REST connector	↩ JSON + HTML của trang	
4	REST connector → Raw staging	lưu bản thô	Cho phép chạy lại bộ phân tích mà không gọi lại Confluence.
Xử lý và lưu			
loop: từng trang trong raw staging			
5	Raw staging → ETL pipeline	trang + checksum_hash	Checksum là MD5 của nội dung trang, dùng để phát hiện thay đổi chứ không dùng cho bảo mật.
6	ETL pipeline → Kho vector	checksum này đã được lưu chưa?	
alt: trang không đổi			
7	Kho vector → ETL pipeline	↩ đã có, bỏ qua	
else: trang mới hoặc đã sửa			
8	Kho vector → ETL pipeline	↩ chưa có	
9	ETL pipeline ⟲	chuyển thành Markdown sạch theo loại nội dung	Alpha: HTML wiki, bảng, ảnh. Beta: Word, PDF, Excel, OCR.

#	Từ → Đến	Điều gì xảy ra	Vì sao / ghi chú
10	ETL pipeline ⟲	cắt thành các đoạn và gắn nhãn	512 token mỗi đoạn, chồng 50. Nhãn: space, phòng ban, loại tệp, mức truy cập, link nguồn, checksum.
11	ETL pipeline → Mô hình embedding	embedding các đoạn	
12	Mô hình embedding → ETL pipeline	↩ các vector	
13	ETL pipeline → Kho vector	upsert văn bản + vector + metadata	
14	Kho vector → ETL pipeline	↩ đã lưu	
15	ETL pipeline → Admin hoặc lịch chạy	↩ tóm tắt: thêm, cập nhật, bỏ qua	Nên giữ trong log để biết lần chạy nào có vấn đề.

Điểm chính

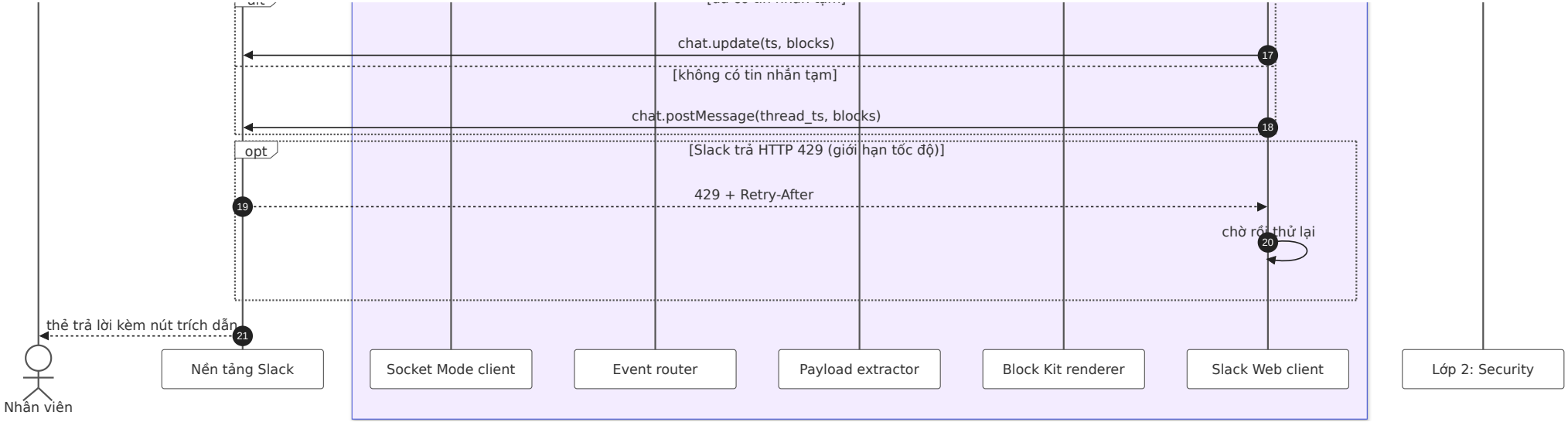
- Bản thô được lưu trước, nên có thể chạy lại bộ phân tích mà không gọi lại Confluence.
- Mã checksum của mỗi trang quyết định có xử lý lại hay không. Trang không đổi được bỏ qua, nên lần chạy thứ hai rất nhanh.
- Nhãn quyền truy cập được gắn cho từng đoạn ở bước này. Nếu sai ở đây thì mọi bộ lọc quyền phía sau đều sai, nên cần kiểm thử kỹ nhất.
- Phải dùng cùng một mô hình embedding ở đây và lúc trả lời câu hỏi, nếu không các vector không so sánh được với nhau.

Sơ đồ 3: Lớp 1: Presentation (giao diện Slack) (1/2)



Lớp 1

Sơ đồ 3: Lớp 1: Presentation (giao diện Slack) (2/2)



Sơ đồ 3: Lớp 1: Presentation (giao diện Slack)

Lớp 1

Mục đích

Bên trong Lớp 1: cách Bolt app nhận một lượt @mention, biến thành gói sự kiện sạch, giao cho Lớp 2, rồi hiển thị kết quả trở lại đúng thread Slack. Lớp 1 không chứa logic nghiệp vụ.

Các đối tượng tham gia

Đối tượng	Vai trò
Nhân viên	Hỏi và đọc câu trả lời
Nền tảng Slack	Sự kiện và Web API (bên ngoài)
Socket Mode client	Giữ kết nối WebSocket
Event router	Lọc và phân phối sự kiện
Payload extractor	Dựng gói EventPayload
Block Kit renderer	Dựng thẻ trả lời
Slack Web client	Đăng và cập nhật tin nhắn
Lớp 2: Security	Lớp kế tiếp

Từng bước

#	Từ → Đến	Điều gì xảy ra	Vì sao / ghi chú
Chiều vào: nhận và chuẩn bị			
1	Nhân viên → Nền tảng Slack	@bot câu hỏi	
2	Nền tảng Slack → Socket Mode client	sự kiện app_mention qua WSS	Bot tự mở kết nối nên Slack chỉ đẩy sự kiện xuống. App-level token cần scope connections:write.
3	Socket Mode client → Event router	chuyển sự kiện	Client tự kết nối lại nếu kết nối rớt.
4	Event router → Nền tảng Slack	↩ ack trong 3 giây	Gửi đầu tiên, trước mọi việc chậm.
alt: sự kiện đến từ chính bot			
5	Event router ⟲	bỏ qua	Tránh vòng lặp bot tự trả lời tin nhắn của mình.
else: người dùng thật nhắc tên bot			
6	Event router → Payload extractor	chuyển sự kiện	
7	Payload extractor → Nền tảng Slack	users.info(user_id)	Cần scope users:read.email. Email là thứ Lớp 2 dùng để nhận diện người hỏi.
8	Nền tảng Slack → Payload extractor	↩ hồ sơ kèm email	
9	Payload extractor ⟲	dựng EventPayload	Các trường: user_id, channel_id, thread_ts, text, email.
opt: bật tin nhắn tạm			
10	Payload extractor → Slack Web client	yêu cầu tin nhắn “đang tìm…”	Cho người dùng phản hồi tức thì trong lúc câu trả lời được chuẩn bị. (Giả định thiết kế.)

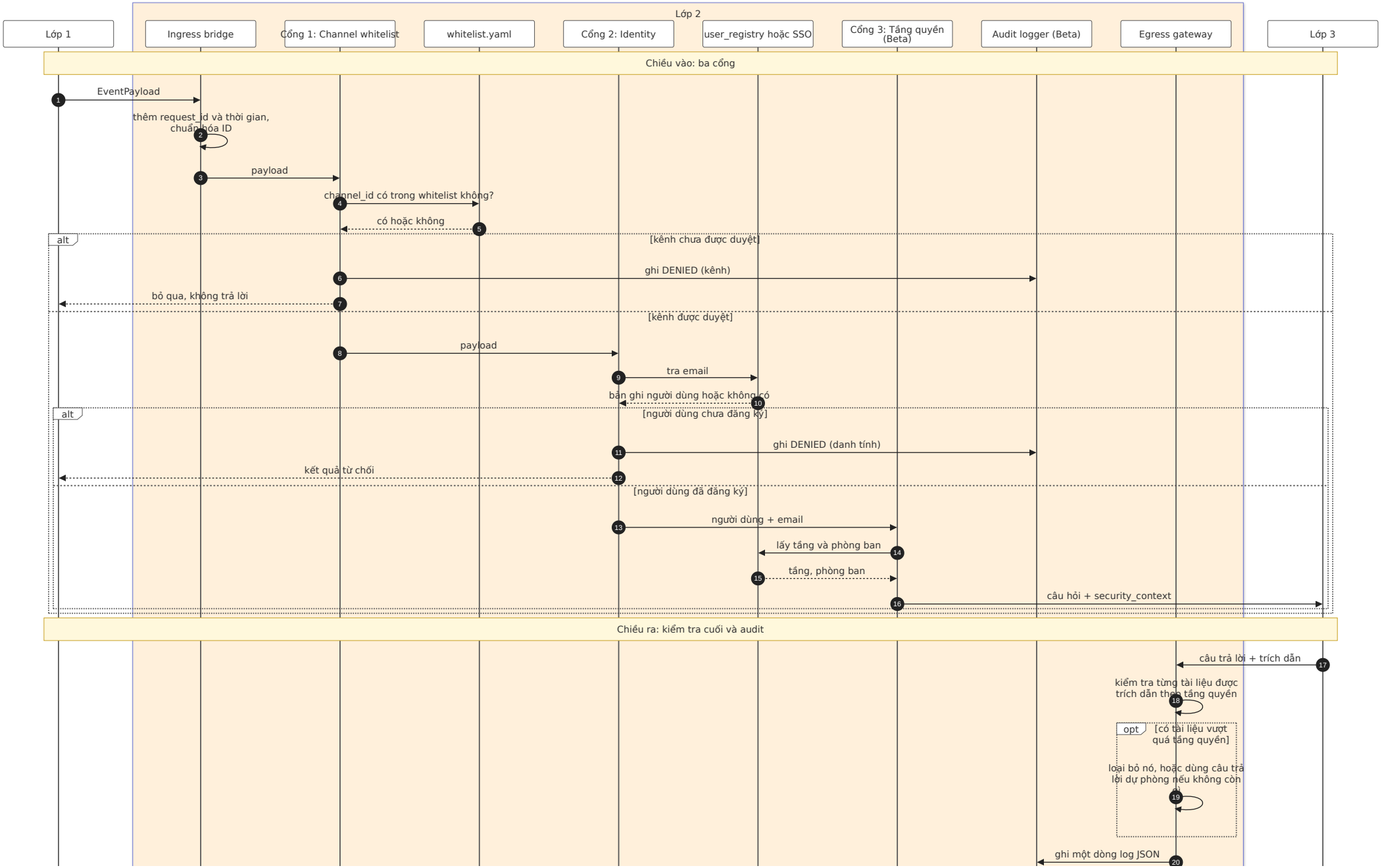
#	Từ → Đến	Điều gì xảy ra	Vì sao / ghi chú
11	Slack Web client → Nền tảng Slack	chat.postMessage trong thread	
12	Nền tảng Slack → Slack Web client	↩ mốc thời gian tin nhắn (ts)	Cần ts để sau này cập nhật chính tin nhắn này.
13	Payload extractor → Lớp 2: Security	EventPayload	Từ đây Lớp 1 chỉ chờ kết quả.
Chiều ra: dựng và đăng			
14	Lớp 2: Security → Block Kit renderer	↩ kết quả: trả lời, dự phòng hoặc từ chối	
15	Block Kit renderer ⟲	chọn kiểu thẻ và dựng blocks	Thẻ trả lời: phần câu trả lời, phần nguồn, mỗi nguồn một nút. Thẻ dự phòng và từ chối đơn giản hơn.
16	Block Kit renderer → Slack Web client	blocks + thread_ts	
alt: đã có tin nhắn tạm			
17	Slack Web client → Nền tảng Slack	chat.update(ts, blocks)	Thay tin nhắn tạm bằng thẻ cuối.
else: không có tin nhắn tạm			
18	Slack Web client → Nền tảng Slack	chat.postMessage(thread_ts, blocks)	Đăng trong thread để kênh gọn gàng.
opt: Slack trả HTTP 429 (giới hạn tốc độ)			
19	Nền tảng Slack → Slack Web client	↩ 429 + Retry-After	
20	Slack Web client ⟲	chờ rồi thử lại	Chờ lùi dần để bot nằm trong giới hạn của Slack.
21	Nền tảng Slack → Nhân viên	↩ thẻ trả lời kèm nút trích dẫn	Mỗi nút mở đúng trang Confluence mà câu trả lời đã dùng.

Điểm chính

- Slack yêu cầu phản hồi xác nhận trong 3 giây. Việc nặng chạy sau khi ack, vì vậy tin nhắn tạm “đang tìm…” giúp người dùng yên tâm.
- Bot bỏ qua tin nhắn của chính nó, tránh vòng lặp tự trả lời.
- Đọc email cần scope users:read.email. Lớp 2 dùng email này để nhận diện người hỏi.
- Role Slack (Owner, member) không quyết định bot hiển thị gì. App Confluence cài trong Slack chỉ để xem trước link, không phải nguồn dữ liệu.

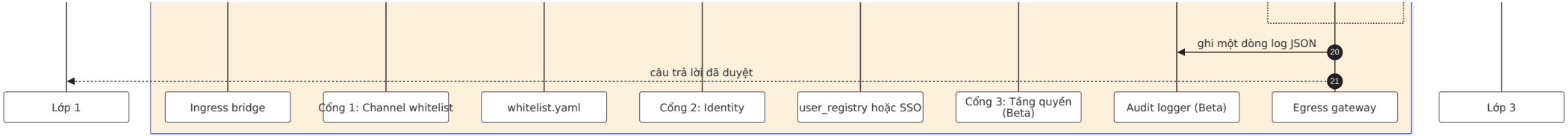
Lớp 2

Sơ đồ 4: Lớp 2: Security và xác thực (1/2)



Lớp 2

Sơ đồ 4: Lớp 2: Security và xác thực (2/2)



Sơ đồ 4: Lớp 2: Security và xác thực

Lớp 2

Mục đích

Bên trong Lớp 2: ba cổng theo thứ tự cố định (kênh, danh tính, tầng quyền), rồi kiểm tra cuối, audit và trả về ở chiều ra. Mặc định là từ chối: một cổng không đạt thì dừng yêu cầu, không có gì đến bước tìm kiếm hay AI.

Các đối tượng tham gia

Đối tượng	Vai trò
Lớp 1	Gửi gói tin, hiển thị kết quả
Ingress bridge	Nhận và chuẩn hóa
Cổng 1: Channel whitelist	Kênh này được duyệt chưa?
whitelist.yaml	Danh sách ID kênh được duyệt (cấu hình)
Cổng 2: Identity	Người này là ai?
user_registry hoặc SSO	Người dùng, tầng quyền, phòng ban
Cổng 3: Tầng quyền (Beta)	Được xem gì?
Audit logger (Beta)	Mỗi yêu cầu một dòng log JSON
Egress gateway	Kiểm tra cuối trước khi trả câu trả lời
Lớp 3	RAG core

Từng bước

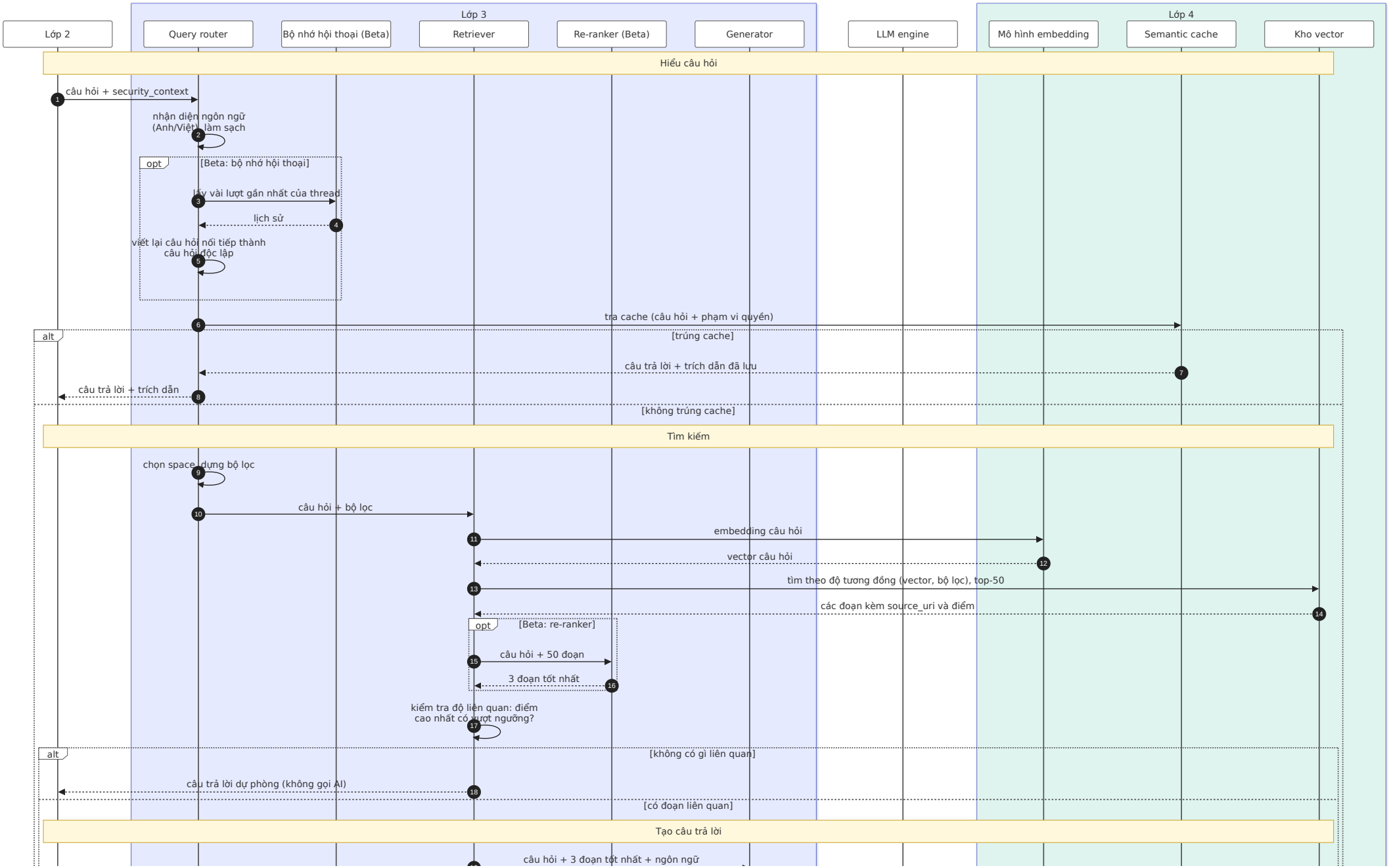
#	Từ → Đến	Điều gì xảy ra	Vì sao / ghi chú
Chiều vào: ba cổng			
1	Lớp 1 → Ingress bridge	EventPayload	
2	Ingress bridge ⟲	thêm request_id và thời gian, chuẩn hóa ID	request_id gắn mọi dòng log của yêu cầu này với nhau.
3	Ingress bridge → Cổng 1: Channel whitelist	payload	
4	Cổng 1: Channel whitelist → whitelist.yaml	channel_id có trong whitelist không?	Cấu hình nằm trong file, không nằm trong code.
5	whitelist.yaml → Cổng 1: Channel whitelist	↩ có hoặc không	
alt: kênh chưa được duyệt			
6	Cổng 1: Channel whitelist → Audit logger (Beta)	ghi DENIED (kênh)	
7	Cổng 1: Channel whitelist → Lớp 1	↩ bỏ qua, không trả lời	Bot im lặng ở những kênh không được phục vụ. (Giả định thiết kế.)
else: kênh được duyệt			
8	Cổng 1: Channel whitelist → Cổng 2: Identity	payload	
9	Cổng 2: Identity → user_registry hoặc SSO	tra email	Alpha: bảng đăng ký do admin duyệt. Project B: SSO của VIB và kiểm tra tên miền @vib.com.vn.
10	user_registry hoặc SSO → Cổng 2: Identity	↩ bản ghi người dùng hoặc không có	

#	Từ → Đến	Điều gì xảy ra	Vì sao / ghi chú
alt: người dùng chưa đăng ký			
11	Cổng 2: Identity → Audit logger (Beta)	ghi DENIED (danh tính)	
12	Cổng 2: Identity → Lớp 1	↩ kết quả từ chối	Lớp 1 hiển thị một lời từ chối ngắn, lịch sự.
else: người dùng đã đăng ký			
13	Cổng 2: Identity → Cổng 3: Tầng quyền (Beta)	người dùng + email	
14	Cổng 3: Tầng quyền (Beta) → user_registry hoặc SSO	lấy tầng và phòng ban	Tầng 1 đến 4. Alpha chỉ dùng tầng 1 và 2. (Đề xuất.)
15	user_registry hoặc SSO → Cổng 3: Tầng quyền (Beta)	↩ tầng, phòng ban	
16	Cổng 3: Tầng quyền (Beta) → Lớp 3	câu hỏi + security_context	security_context = user_id, email, tầng, phòng ban, request_id.
Chiều ra: kiểm tra cuối và audit			
17	Lớp 3 → Egress gateway	câu trả lời + trích dẫn	
18	Egress gateway ⟲	kiểm tra từng tài liệu được trích dẫn theo tầng quyền	So mức truy cập và phòng ban của tài liệu với người hỏi.
opt: có tài liệu vượt quá tầng quyền			
19	Egress gateway ⟲	loại bỏ nó, hoặc dùng câu trả lời dự phòng nếu không còn gì	
20	Egress gateway → Audit logger (Beta)	ghi một dòng log JSON	Thời gian, email, tầng, câu hỏi, loại tài liệu, APPROVED hoặc DENIED.
21	Egress gateway → Lớp 1	↩ câu trả lời đã duyệt	

Điểm chính

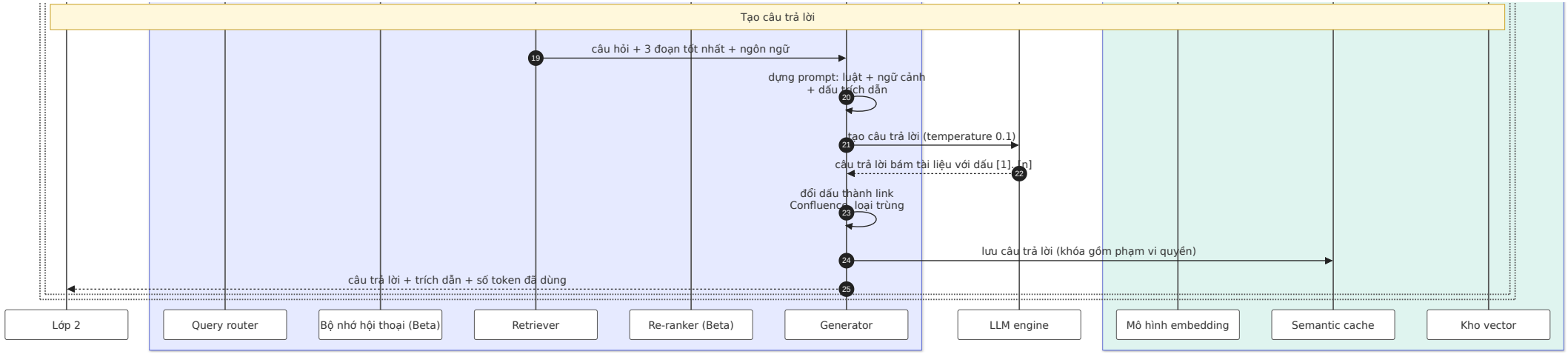
- Thứ tự là rẻ nhất trước: kênh sai bị loại trước khi tra cơ sở dữ liệu.
- Danh tính ở Alpha là bảng đăng ký do admin duyệt. Project B thay bước tra này bằng đăng nhập một lần của VIB. Bảng tầng quyền giữ nguyên.
- Đầu ra là security_context. Lớp 3 biến nó thành bộ lọc cơ sở dữ liệu, nên quyền được áp dụng trước khi AI nhìn thấy bất kỳ văn bản nào.
- Kiểm tra ở chiều ra là lớp phòng thủ thứ hai: dù phần tìm kiếm có sai, không có gì vượt quá tầng của người hỏi rời khỏi Lớp 2.

Sơ đồ 5: Lớp 3: RAG core và logic nghiệp vụ (1/2)



Lớp 3

Sơ đồ 5: Lớp 3: RAG core và logic nghiệp vụ (2/2)



Sơ đồ 5: Lớp 3: RAG core và logic nghiệp vụ

Lớp 3

Mục đích

Bên trong Lớp 3: câu hỏi được hiểu, đối chiếu với cache, biến thành một lượt tìm có bộ lọc, có thể xếp hạng lại, rồi hoặc được trả lời kèm nguồn hoặc được trả lời dự phòng an toàn. Các bước chia thành hiểu, tìm và tạo câu trả lời.

Các đối tượng tham gia

Đối tượng	Vai trò
Lớp 2	Gửi câu hỏi và security_context
Query router	Ngôn ngữ, bộ lọc, kiểm tra cache
Bộ nhớ hội thoại (Beta)	Vài lượt gần nhất của thread
Retriever	Tìm kiếm và kiểm tra độ liên quan
Re-ranker (Beta)	Giữ ba đoạn tốt nhất
Generator	Prompt, trích dẫn
LLM engine	Ollama hoặc Bedrock (adapter)
Mô hình embedding	Đổi câu hỏi thành vector
Semantic cache	Câu trả lời đã lưu
Kho vector	Tìm tương đồng có bộ lọc

Từng bước

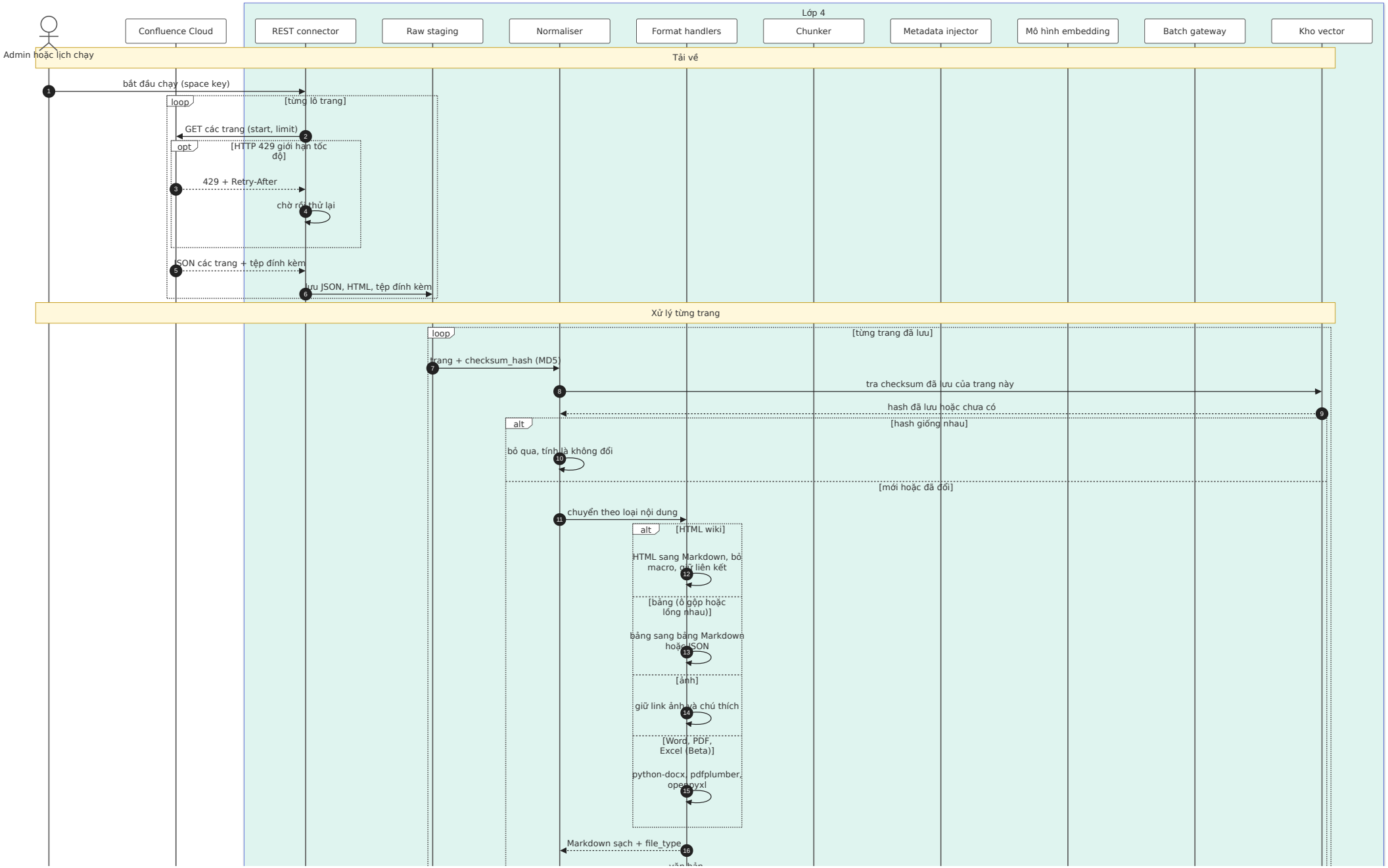
#	Từ → Đến	Điều gì xảy ra	Vì sao / ghi chú
Hiểu câu hỏi			
1	Lớp 2 → Query router	câu hỏi + security_context	
2	Query router ⟲	nhận diện ngôn ngữ (Anh/Việt), làm sạch	Bỏ phần @mention, mở rộng viết tắt. Mã ngôn ngữ đi kèm yêu cầu.
opt: Beta: bộ nhớ hội thoại			
3	Query router → Bộ nhớ hội thoại (Beta)	lấy vài lượt gần nhất của thread	
4	Bộ nhớ hội thoại (Beta) → Query router	↩ lịch sử	
5	Query router ⟲	viết lại câu hỏi nối tiếp thành câu hỏi độc lập	Ví dụ “còn nghỉ ốm thì sao?” thành một câu hỏi đầy đủ.
6	Query router → Semantic cache	tra cache (câu hỏi + phạm vi quyền)	Phạm vi quyền là một phần của khóa. (Giả định thiết kế.)
alt: trúng cache			
7	Semantic cache → Query router	↩ câu trả lời + trích dẫn đã lưu	
8	Query router → Lớp 2	↩ câu trả lời + trích dẫn	Bỏ qua tìm kiếm và lệnh gọi AI.
else: không trúng cache			
Tìm kiếm			
9	Query router ⟲	chọn space, dựng bộ lọc	Các token lọc: space_key, department_tag, access_level ≤ tầng.
10	Query router → Retriever	câu hỏi + bộ lọc	

#	Từ → Đến	Điều gì xảy ra	Vì sao / ghi chú
11	Retriever → Mô hình embedding	embedding câu hỏi	Phải là cùng mô hình đã embedding các tài liệu.
12	Mô hình embedding → Retriever	↩ vector câu hỏi	
13	Retriever → Kho vector	tìm theo độ tương đồng (vector, bộ lọc), top-50	Đi qua retrieval gateway (sơ đồ 7).
14	Kho vector → Retriever	↩ các đoạn kèm source_uri và điểm	
opt: Beta: re-ranker			
15	Retriever → Re-ranker (Beta)	câu hỏi + 50 đoạn	
16	Re-ranker (Beta) → Retriever	↩ 3 đoạn tốt nhất	Một mô hình chậm hơn nhưng chính xác hơn chấm lại 50 ứng viên.
17	Retriever ⟲	kiểm tra độ liên quan: điểm cao nhất có vượt ngưỡng?	Khi chưa có re-ranker thì dùng ba đoạn điểm cao nhất.
alt: không có gì liên quan			
18	Retriever → Lớp 2	↩ câu trả lời dự phòng (không gọi AI)	Nói bằng ngôn ngữ của người hỏi. Không cache.
else: có đoạn liên quan			
Tạo câu trả lời			
19	Retriever → Generator	câu hỏi + 3 đoạn tốt nhất + ngôn ngữ	
20	Generator ⟲	dựng prompt: luật + ngữ cảnh + dấu trích dẫn	Luật: chỉ dùng ngữ cảnh, trích dẫn mọi ý bằng [1] đến [n], trả lời theo ngôn ngữ câu hỏi.
21	Generator → LLM engine	tạo câu trả lời (temperature 0.1)	Nhiệt độ thấp giữ câu trả lời ổn định và bám tài liệu.
22	LLM engine → Generator	↩ câu trả lời bám tài liệu với dấu [1]..[n]	
23	Generator ⟲	đổi dấu thành link Confluence, loại trùng	source_uri thành link bấm được, xếp theo độ liên quan.
24	Generator → Semantic cache	lưu câu trả lời (khóa gồm phạm vi quyền)	
25	Generator → Lớp 2	↩ câu trả lời + trích dẫn + số token đã dùng	

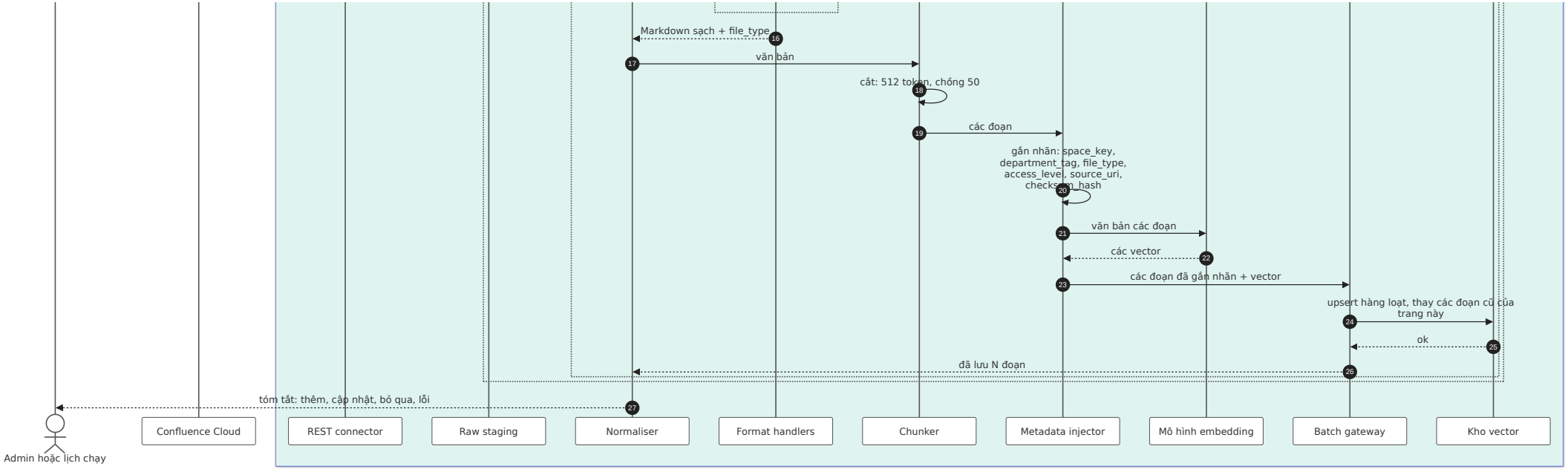
Điểm chính

- Bộ lọc được dựng từ security_context trước khi bất kỳ lượt tìm nào chạy, nên quyền của người hỏi định hình ngay câu truy vấn.
- Khóa cache phải gồm cả phạm vi quyền của người dùng. Nếu không, câu trả lời dựng cho quản lý có thể hiện cho nhân viên. (Giả định thiết kế, cần nhóm xác nhận.)
- Nếu không đoạn nào vượt ngưỡng điểm thì lớp này trả lời dự phòng và không gọi AI, nên không thể bịa câu trả lời.
- LLM, kho vector, mô hình embedding và nguồn dữ liệu đều dùng qua adapter. Đổi Ollama (Project A) sang Bedrock (Project B) chỉ là đổi cấu hình.
- Phần Beta: bộ nhớ hội thoại và re-ranker. Khi chưa có thì dùng ba đoạn điểm cao nhất.

Sơ đồ 6: Lớp 4: Nạp dữ liệu chi tiết (offline) (1/2)



Sơ đồ 6: Lớp 4: Nạp dữ liệu chi tiết (offline) (2/2)



Sơ đồ 6: Lớp 4: Nạp dữ liệu chi tiết (offline)

Lớp 4

Mục đích

Bản chi tiết của sơ đồ 2: cách một trang được tải về có phân trang và thử lại, đối chiếu với checksum đã lưu, chuyển đến bộ xử lý đúng định dạng, cắt thành đoạn, gắn nhãn, embedding và ghi vào kho.

Các đối tượng tham gia

Đối tượng	Vai trò
Admin hoặc lịch chạy	Khởi động lần chạy
Confluence Cloud	Nguồn (bên ngoài)
REST connector	Phân trang, thử lại
Raw staging	Thư mục local hoặc S3
Normaliser	Kiểm tra checksum, điều phối
Format handlers	Mỗi loại nội dung một bộ xử lý
Chunker	512 token, chồng 50
Metadata injector	Gắn nhãn
Mô hình embedding	Văn bản thành vector
Batch gateway	Ghi hàng loạt
Kho vector	PostgreSQL + pgvector

Từng bước

#	Từ → Đến	Điều gì xảy ra	Vì sao / ghi chú
Tải về			
1	Admin hoặc lịch chạy → REST connector	bắt đầu chạy (space key)	
loop: từng lô trang			
2	REST connector → Confluence Cloud	GET các trang (start, limit)	Dùng API token lấy từ biến môi trường.
opt: HTTP 429 giới hạn tốc độ			
3	Confluence Cloud → REST connector	↩ 429 + Retry-After	
4	REST connector ⟲	chờ rồi thử lại	
5	Confluence Cloud → REST connector	↩ JSON các trang + tệp đính kèm	
6	REST connector → Raw staging	lưu JSON, HTML, tệp đính kèm	Project A: thư mục /data/raw_dumps. Project B: S3 mã hóa SSE-S3.
Xử lý từng trang			
loop: từng trang đã lưu			
7	Raw staging → Normaliser	trang + checksum_hash (MD5)	
8	Normaliser → Kho vector	tra checksum đã lưu của trang này	
9	Kho vector → Normaliser	↩ hash đã lưu hoặc chưa có	
alt: hash giống nhau			
10	Normaliser ⟲	bỏ qua, tính là không đổi	

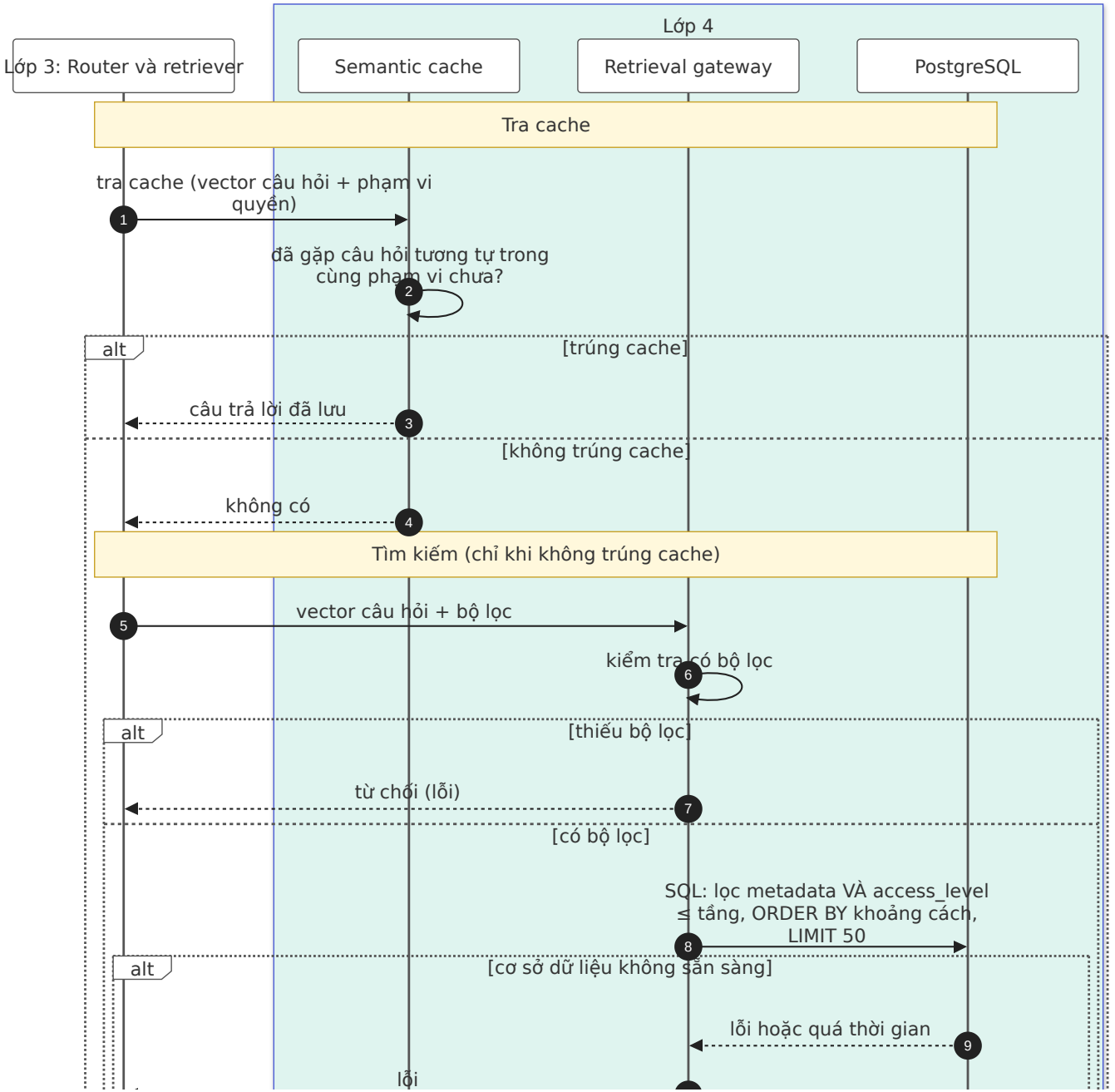
#	Từ → Đến	Điều gì xảy ra	Vì sao / ghi chú
else: mới hoặc đã đổi			
11	Normaliser → Format handlers	chuyển theo loại nội dung	
alt: HTML wiki			
12	Format handlers ⟲	HTML sang Markdown, bỏ macro, giữ liên kết	BeautifulSoup4.
else: bảng (ô gộp hoặc lồng nhau)			
13	Format handlers ⟲	bảng sang bảng Markdown hoặc JSON	Giữ con số gắn với tiêu đề cột. html5lib, pandas, Unstructured.
else: ảnh			
14	Format handlers ⟲	giữ link ảnh và chú thích	OCR ảnh chụp màn hình là Beta.
else: Word, PDF, Excel (Beta)			
15	Format handlers ⟲	python-docx, pdfplumber, openpyxl	Tệp .doc cũ được chuyển bằng LibreOffice trước.
16	Format handlers → Normaliser	↩ Markdown sạch + file_type	
17	Normaliser → Chunker	văn bản	
18	Chunker ⟲	cắt: 512 token, chồng 50	Phần chồng giúp không mất ý ở chỗ cắt.
19	Chunker → Metadata injector	các đoạn	
20	Metadata injector ⟲	gắn nhãn: space_key, department_tag, file_type, access_level, source_uri, checksum_hash	Bước quan trọng nhất cho bảo mật.
21	Metadata injector → Mô hình embedding	văn bản các đoạn	
22	Mô hình embedding → Metadata injector	↩ các vector	Kích thước vector phải khớp cột trong cơ sở dữ liệu.
23	Metadata injector → Batch gateway	các đoạn đã gắn nhãn + vector	
24	Batch gateway → Kho vector	upsert hàng loạt, thay các đoạn cũ của trang này	Ghi hàng loạt psycopg3 và SQLAlchemy (COPY).
25	Kho vector → Batch gateway	↩ ok	
26	Batch gateway → Normaliser	↩ đã lưu N đoạn	
27	Normaliser → Admin hoặc lịch chạy	↩ tóm tắt: thêm, cập nhật, bỏ qua, lỗi	

Điểm chính

- Thử lại và raw staging bảo vệ Confluence khỏi các lệnh gọi lặp lại khi nhóm gỡ lỗi bộ phân tích.
- Mỗi loại nội dung có bộ xử lý riêng. Alpha gồm trang wiki, bảng phức tạp và ảnh. Word, PDF, Excel và OCR thuộc Beta.
- Nhãn được gắn trước khi embedding, nên mỗi dòng trong kho mang theo mức truy cập của chính nó.
- Trang được sửa sẽ thay thế các đoạn cũ của nó, nếu không cả văn bản cũ và mới đều tìm được. (Giả định thiết kế.)

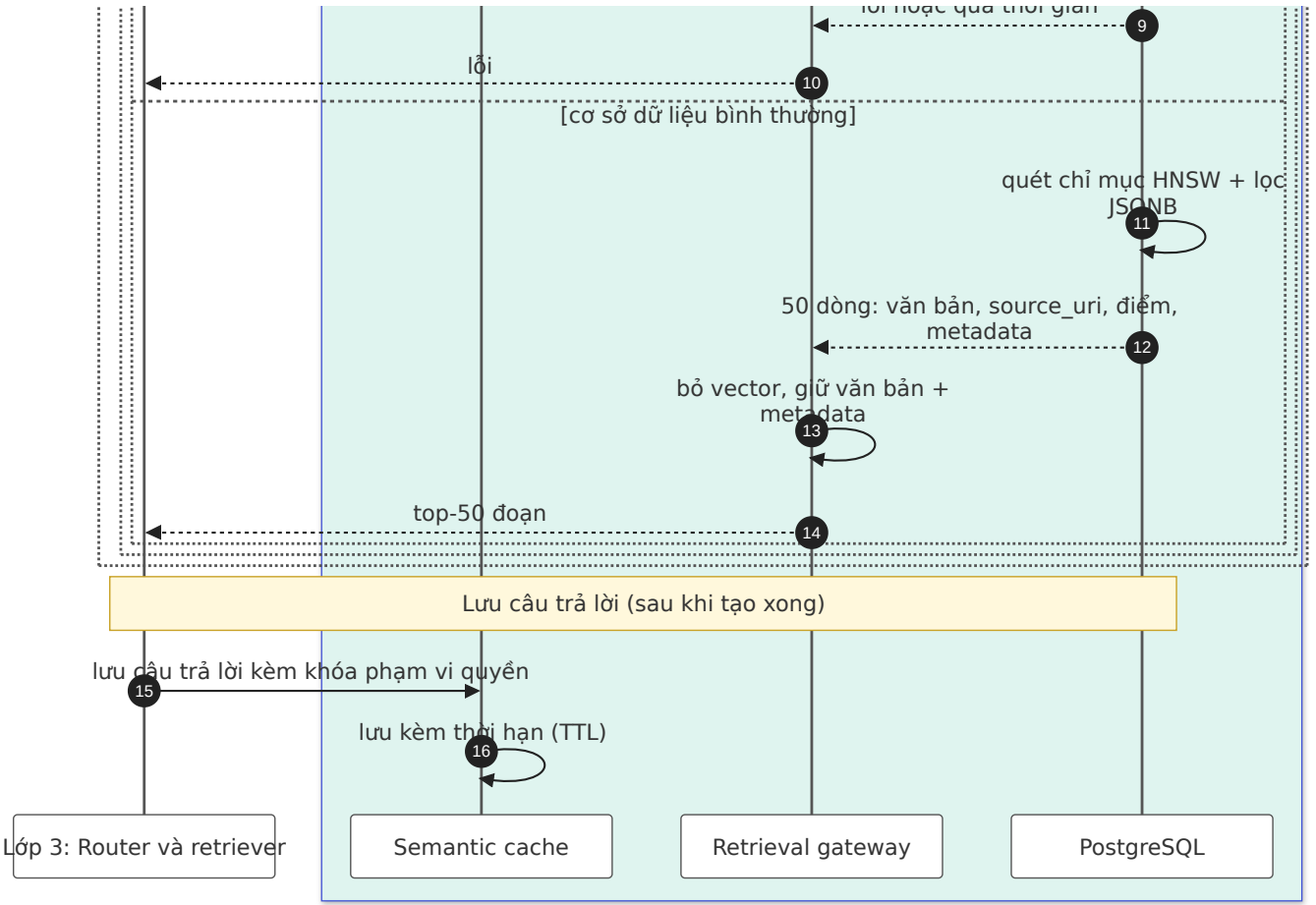
- Khoảng trống cần quyết định: trang bị xóa trong Confluence để lại các đoạn mồ côi. Có thể kiểm tra định kỳ danh sách ID trang để dọn. (Đề xuất.)

Sơ đồ 7: Lớp 4: Tìm kiếm và cache lúc có câu hỏi (1/2)



Lớp 4

Sơ đồ 7: Lớp 4: Tìm kiếm và cache lúc có câu hỏi (2/2)



Sơ đồ 7: Lớp 4: Tìm kiếm và cache lúc có câu hỏi

Lớp 4

Mục đích

Bên trong Lớp 4 khi có câu hỏi: kiểm tra cache, retrieval gateway bắt buộc có bộ lọc quyền, PostgreSQL chạy một truy vấn kết hợp bộ lọc với độ tương đồng vector, và sau đó câu trả lời được lưu lại vào cache.

Các đối tượng tham gia

Đối tượng	Vai trò
Lớp 3: Router và retriever	Hỏi và nhận các đoạn
Semantic cache	Redis hoặc dict trong bộ nhớ
Retrieval gateway	Kiểm tra và chạy truy vấn
PostgreSQL	pgvector (HNSW) + metadata JSONB

Từng bước

#	Từ → Đến	Điều gì xảy ra	Vì sao / ghi chú
Tra cache			
1	Lớp 3: Router và retriever → Semantic cache	tra cache (vector câu hỏi + phạm vi quyền)	
2	Semantic cache ⟲	đã gặp câu hỏi tương tự trong cùng phạm vi chưa?	So vector với các vector đã lưu trong một ngưỡng tương đồng.
alt: trúng cache			
3	Semantic cache → Lớp 3: Router và retriever	↩ câu trả lời đã lưu	Trả ngay nên phản hồi rất nhanh.
else: không trúng cache			
4	Semantic cache → Lớp 3: Router và retriever	↩ không có	
Tìm kiếm (chỉ khi không trúng cache)			
5	Lớp 3: Router và retriever → Retrieval gateway	vector câu hỏi + bộ lọc	Bộ lọc: space_key, department_tag, access_level ≤ tầng.
6	Retrieval gateway ⟲	kiểm tra có bộ lọc	Lượt tìm không có bộ lọc quyền sẽ bị từ chối.
alt: thiếu bộ lọc			
7	Retrieval gateway → Lớp 3: Router và retriever	↩ từ chối (lỗi)	Đóng khi có lỗi.
else: có bộ lọc			
8	Retrieval gateway → PostgreSQL	SQL: lọc metadata VÀ access_level ≤ tầng, ORDER BY khoảng cách, LIMIT 50	Một truy vấn làm cả lọc quyền và xếp hạng tương đồng.
alt: cơ sở dữ liệu không sẵn sàng			
9	PostgreSQL → Retrieval gateway	↩ lỗi hoặc quá thời gian	
10	Retrieval gateway → Lớp 3: Router và retriever	↩ lỗi	Lớp 3 hiện thông báo thân thiện và sự cố được ghi log.
else: cơ sở dữ liệu bình thường			

#	Từ → Đến	Điều gì xảy ra	Vì sao / ghi chú
11	PostgreSQL ⟲	quét chỉ mục HNSW + lọc JSONB	Chỉ mục tìm các điểm gần nhất rất nhanh, trong vài mili-giây.
12	PostgreSQL → Retrieval gateway	↩ 50 dòng: văn bản, source_uri, điểm, metadata	
13	Retrieval gateway ⟲	bỏ vector, giữ văn bản + metadata	Vector nặng và Lớp 3 không cần.
14	Retrieval gateway → Lớp 3: Router và retriever	↩ top-50 đoạn	
Lưu câu trả lời (sau khi tạo xong)			
15	Lớp 3: Router và retriever → Semantic cache	lưu câu trả lời kèm khóa phạm vi quyền	Chỉ lưu câu trả lời có nguồn.
16	Semantic cache ⟲	lưu kèm thời hạn (TTL)	Thời hạn ngăn câu trả lời cũ tồn tại mãi sau khi trang thay đổi. (Giả định thiết kế.)

Điểm chính

- Bảo mật và độ tương đồng được áp dụng trong cùng một truy vấn vì vector, văn bản và nhãn nằm trong cùng một dòng.
- Gateway đóng khi có lỗi: không bao giờ chạy lượt tìm không có bộ lọc. (Giả định thiết kế.)
- Nếu cơ sở dữ liệu không sẵn sàng, gateway trả lỗi và Lớp 3 hiện thông báo “hãy thử lại” thân thiện thay vì bị sập.
- Bản nháp của nhóm còn nhắc đến cache kết quả tìm kiếm. Sơ đồ này cache câu trả lời cuối cùng, tiết kiệm cả lệnh gọi AI. Cần chọn một cách trong decision record.

Các giả định thiết kế cần nhóm xác nhận

Các sơ đồ có những lựa chọn sau đây mà Charter và bản draw.io của nhóm chưa nêu. Đây chỉ là đề xuất. Hãy xác nhận hoặc sửa trước khi chúng trở thành yêu cầu.

1. Kênh lạ bị bỏ qua im lặng. Người bị từ chối nhận một thông báo ngắn, lịch sự.
2. Khóa cache gồm phạm vi quyền của người dùng (tầng và phòng ban). Cần thiết để câu trả lời của tầng này không hiện cho tầng khác.
3. Một tin nhắn tạm “đang tìm…” hiện ngay và được thay bằng câu trả lời (Lớp 1).
4. Giới hạn tốc độ của Slack (HTTP 429) được xử lý bằng chờ rồi thử lại.
5. Retrieval gateway không bao giờ chạy lượt tìm khi thiếu bộ lọc quyền (đóng khi có lỗi).
6. Trang Confluence được sửa sẽ thay các đoạn cũ của nó. Trang bị xóa được dọn bằng kiểm tra định kỳ (đề xuất).
7. Câu trả lời trong cache hết hạn sau một khoảng thời gian (TTL).
8. Ánh xạ tầng quyền: project manager tầng 4, dev tầng 2, client tầng 1 (đề xuất). Alpha dùng tầng 1 và 2.
9. Các phần Beta (kiểm tra tầng quyền, audit log, re-ranker, bộ nhớ hội thoại, bộ xử lý Word/PDF/Excel/OCR) được vẽ để có thể tắt mà luồng Alpha vẫn chạy.